

Programmering med mikrokontroller

Aut2602 Innlevering 3

Jens Christian Valen Leynse

Oktober 2024

1 Kode segmenter

Denne header-filen deklarerer funksjonene som brukes til å konfigurere timer.
timer.h

```
1 #ifndef TIMER_H
2 #define TIMER_H
3
4 #include <avr/io.h>
5 #include <avr/interrupt.h>
6 #include <util/delay.h>
7
8 #define F_CPU 16000000UL
9
10 // Function to initialize the main clock to 16
    MHz
11 void clock_init();
12
13 // Function to initialize Timer/Counter A in
    basic mode
14 void tca_init_basic_mode();
15
16 // Function to blink an LED at 1000 Hz
17 void blink_led_1000Hz();
18
19 // Function to blink an LED at 1 Hz
20 void blink_led_1Hz();
21
```

```

22 // Function to gradually increase frequency
    between 1 kHz and 5 kHz
23 void increase_buzzer_frequency();
24
25 // Function to gradually decrease frequency
26 void decrease_buzzer_frequency();
27
28 // Function that simulates highest possible
    frequency
29 void highest_frequency();
30
31 // Function that simulates lowest possible
    frequency
32 void lowest_frequency();
33
34 #endif // TIMER_H

```

timer.c

```

1 #include "timer.h"
2
3 // Function to configure the main clock to use
    16 MHz
4 void clock_init () {
5     ccp_write_io (( void *) (CLKCTRL.
        OSCHFCTRLA) , CLKCTRL_FRQSEL_16M_gc |
        CLKCTRL_AUTOTUNE_bm |
        CLKCTRL_RUNSTDBY_bm);
6 }
7
8 // Function to initialize Timer/Counter A in
    basic mode
9 void tca_init_basic_mode() {
10     TCA0.SINGLE.CTRLA =
        TCA_SINGLE_CLKSEL_DIV16_gc |
        TCA_SINGLE_ENABLE_bm; // Set prescaler
        to 64
11     TCA0.SINGLE.PER = 1000; // Set period for
        1 ms overflow (1000 Hz)

```

```

12     TCA0.SINGLE.INTCTRL = TCA_SINGLE_OVF_bm;
        // Enable overflow interrupt
13 }
14
15 // Function to blink an LED at 1000 Hz
16 void blink_led_1000Hz() {
17     PORTB.OUTTGL = PIN3_bm; // Toggle LED on
        PIN3
18 }
19
20 // Function to blink an LED at 1 Hz
21 void blink_led_1Hz() {
22     PORTB.OUTTGL = PIN3_bm; // Toggle LED on
        PIN3
23     TCA0.SINGLE.PER = 100000; // Change period
        for 1 Hz //mulig 1000000
24 }
25
26 // Interrupt Service Routine for Timer/Counter
    A overflow
27 ISR(TCA0_OVF_vect) {
28     PORTF.OUTTGL = PIN2_bm;
29     PORTB.OUTTGL = PIN3_bm;
30     TCA0.SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm;
31 }
32
33 // Function to gradually increase frequency
    between 1 kHz and 5 kHz
34 void increase_buzzer_frequency() {
35     for (uint16_t freq = 1000; freq <= 5000;
        freq += 50) {
36         TCA0.SINGLE.PER = (F_CPU / 16 / freq);
            // Set buzzer frequency
37         _delay_ms(100); // Delay to observe
            frequency change
38     }
39 }
40
41 // The highest frequency that an average human

```

```

    can hear typically ranges between 20 Hz
    and 20 kHz (20,000 Hz)
42 void highest_frequency() {
43     TCA0.SINGLE.PER = (F_CPU / 16 / 142857);
        // Set buzzer frequency
44     _delay_ms(100); // Delay to observe
        frequency change
45 }
46
47 void lowest_frequency() { // hard to determine
    as it lies between frequent clicks and
    constant noise
48     TCA0.SINGLE.PER = (F_CPU / 16 / 250); //
        Set buzzer frequency
49     _delay_ms(100); // Delay to observe
        frequency change
50 }
51
52 // Function to gradually decrease frequency
53 void decrease_buzzer_frequency() {
54     for (uint16_t freq = 5000; freq >= 1000;
        freq -= 50) {
55         TCA0.SINGLE.PER = (F_CPU / 16 / freq);
            // Set buzzer frequency
56         _delay_ms(100); // Delay to observe
            frequency change
57     }
58 }

```

main.c

```

1 #include "timer.h"
2
3 int main(void) {
4     PORTB.DIRSET = PIN3_bm; // Set PIN3 as
        output for LED
5     PORTF.DIRSET = PIN2_bm;
6
7     clock_init();
8     tca_init_basic_mode();

```

```

9
10 blink_led_1Hz();
11 //blink_led_1000Hz();
12
13 sei(); // Enable global interrupt
14
15 while (1) {
16     // Gradually increase frequency
17     increase_buzzer_frequency();
18
19     // Gradually decrease frequency
20     decrease_buzzer_frequency();
21
22     // Highest frequency
23     highest_frequency();
24
25     // Lowest frequency
26     lowest_frequency();
27 }
28 }

```